

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

FACULTAD DE SISTEMAS E INFORMÁTICA



REQUISITOS DE DESARROLLO

ASIGNATURA

Ingeniería de requisitos

DOCENTE

RODRÍGUEZ RODRÍGUEZ , CIRO

ALUMNO:

Cordova Guerra, Josue Rodrigo

Melendez Bustamante, Alvaro Mathias

Alva Chacon, Jose

Sandoval Dominguez, Erick Marco

2026

1. Introducción

Durante la fase de identificación y análisis de requisitos de desarrollo del proyecto OLYMPUS CORE, el equipo evaluó las restricciones técnicas, metodológicas y operativas necesarias para garantizar la construcción de una plataforma robusta, segura, escalable y alineada a las necesidades del gimnasio.

El análisis permitió identificar factores críticos relacionados con concurrencia en reservas de clases, validación de acceso mediante código QR dinámico, rendimiento del sistema, seguridad de la información, compatibilidad tecnológica y despliegue en infraestructura de producción realista. Asimismo, se definieron estrategias de mitigación de riesgos y mecanismos de validación técnica orientados a garantizar la calidad del producto final.

El desarrollo del proyecto se basa en metodologías ágiles (Extreme Programming) y herramientas estándar de la industria, priorizando la construcción incremental del MVP1, la trazabilidad de cambios y la validación continua de requisitos. El stack tecnológico seleccionado corresponde a tecnologías de uso real en producción: Node.js, PostgreSQL, React y Nginx sobre un VPS Linux, asegurando que la plataforma pueda operar de forma autónoma y segura en el entorno del gimnasio.

2. Identificación y Análisis de Requisitos de Desarrollo

2.1. Mapeo de Restricciones del Proyecto

Durante esta etapa se identificaron las restricciones técnicas, temporales, metodológicas y operativas que condicionan el desarrollo del sistema:

Tipo de restricción	Descripción de la restricción	Justificación / Fuente
Tecnológica	El sistema requiere hardware específico en recepción: lectores de códigos QR (smartphones o lectores dedicados) y torniquete físico con interfaz HTTP o serial.	Análisis de requisitos del gimnasio — RD-01, RF-06
Tecnológica	El SGBD será estrictamente PostgreSQL 16 para asegurar integridad relacional, soporte ACID, transacciones con SELECT FOR UPDATE y manejo eficiente de concurrencia.	Decisión técnica del equipo — RNF-07, TEC-001
Tecnológica	El servidor de producción será un VPS Linux (Ubuntu 22.04 LTS) gestionado con Nginx como reverse proxy y PM2 como process manager para Node.js. Se excluyen plataformas PaaS gratuitas	Requisito de disponibilidad — RQ-CAL-OLY-11 (99% uptime)

	(Heroku free tier, etc.) por limitaciones de tiempo de inactividad y recursos.	
Tecnológica	La autenticación se implementará con JWT (HS256) y bcrypt (factor 12). Las sesiones duran 24 horas y los tokens QR expiran en 12 horas. No se almacenan contraseñas en texto plano.	RNF-04, RNF-05, RNF-06 — Seguridad
Temporal	El tiempo máximo para el desarrollo y entrega del MVP1 está fijado en 5 meses. Las iteraciones semanales de XP garantizan entregables funcionales al final de cada sprint.	Límite académico del proyecto
Proceso	El desarrollo se gestionará mediante Extreme Programming (XP): Pair Programming, TDD, integración continua con GitHub Actions, Planning Game semanal y cliente in-situ.	Metodología de desarrollo seleccionada
Seguridad	Todas las comunicaciones deben ir sobre HTTPS (TLS 1.2 mínimo) con certificado SSL válido (Let's Encrypt). El tráfico HTTP debe ser redirigido automáticamente a HTTPS.	RNF-05, RQ-CAL-OLY-05
Rendimiento	La validación de acceso QR debe completarse en menos de 300 ms (P95). La respuesta general de la API debe ser < 500 ms (P95). Se requieren índices en columnas críticas de la BD.	RNF-01, RNF-02 — RQ-CAL-OLY-01, RQ-CAL-OLY-02
Concurrencia	El sistema debe soportar mínimo 100 usuarios concurrentes sin sobreventa de cupos en reservas. Se implementa SELECT FOR UPDATE NOWAIT en PostgreSQL.	Shadowing horas pico — RNF-03, RQ-CAL-OLY-03
Escalabilidad	La arquitectura modular (separación frontend/backend/BD) debe permitir escalar de forma	Stakeholders — RD-03, versión 2.0 y 3.0

	independiente cada capa sin rediseñar el sistema completo.	
Compatibilidad	La interfaz web debe ser funcional en Chrome, Firefox, Edge y Safari (últimas 2 versiones). Compatible con pantallas $\geq 768\text{px}$ (tablets de recepción).	RNF-08, RNF-09 — RQ-CAL-OLY-08, RQ-CAL-OLY-09
Integración	El sistema debe permitir futuras integraciones con APIs externas (pasarela de pagos — SC-002, WhatsApp — SC-001) mediante diseño de endpoints REST versionados.	Dueño del gimnasio — Gestión de Cambios v2.0
Legal	El registro de datos personales (DNI, teléfono, historial médico) debe cumplir con la Ley N.º 29733 de Protección de Datos Personales del Perú. Se requiere consentimiento explícito del socio.	Requisito legal del dominio — RD-05

La siguiente tabla clasifica los elementos del proyecto según su naturaleza:

Elemento	Clasificación	Justificación
Desarrollo iterativo en sprints de 1 semana	Requisito de Proyecto / Proceso	Define la frecuencia de hitos de validación y la metodología de trabajo. No visible para el usuario final.
PostgreSQL con transacciones ACID	Requisito de Proyecto (Restricción técnica)	Condiciona el motor de BD y la estrategia de concurrencia. Impacta en la arquitectura de toda la capa de datos.
VPS Linux + Nginx + PM2	Requisito de Proyecto (Restricción de infraestructura)	Define el entorno de despliegue de producción. Limita las decisiones de hosting del equipo.
Integración obligatoria con torniquete físico	Requisito de Proyecto (Restricción técnica)	Obliga al equipo a adaptarse al hardware preexistente del gimnasio (protocolo HTTP/serial).
Control de versiones con Git y GitHub	Requisito de Proyecto / Proceso	Herramienta exclusiva del equipo para gestión del código fuente y CI/CD.
Validación de código QR en menos de 300 ms	Requisito del Producto (RNF)	Comportamiento del sistema de cara al usuario

		(recepcionista y socio). Verificable con métricas.
Registro de Ficha Médica bajo Ley N.º 29733	Requisito de Dominio / Restricción Legal	Normativa peruana que el producto debe cumplir para operar legalmente. Impacta en formularios y consentimientos.
JWT con expiración de 24h / QR con 12h	Requisito de Producto (RNF-06)	Comportamiento de seguridad verificable técnicamente con herramientas de decodificación JWT.

El siguiente diagrama muestra el stack tecnológico de producción elegido y la relación entre sus capas:

Stack Tecnológico de Producción — OLYMPUS CORE (Capas del sistema)

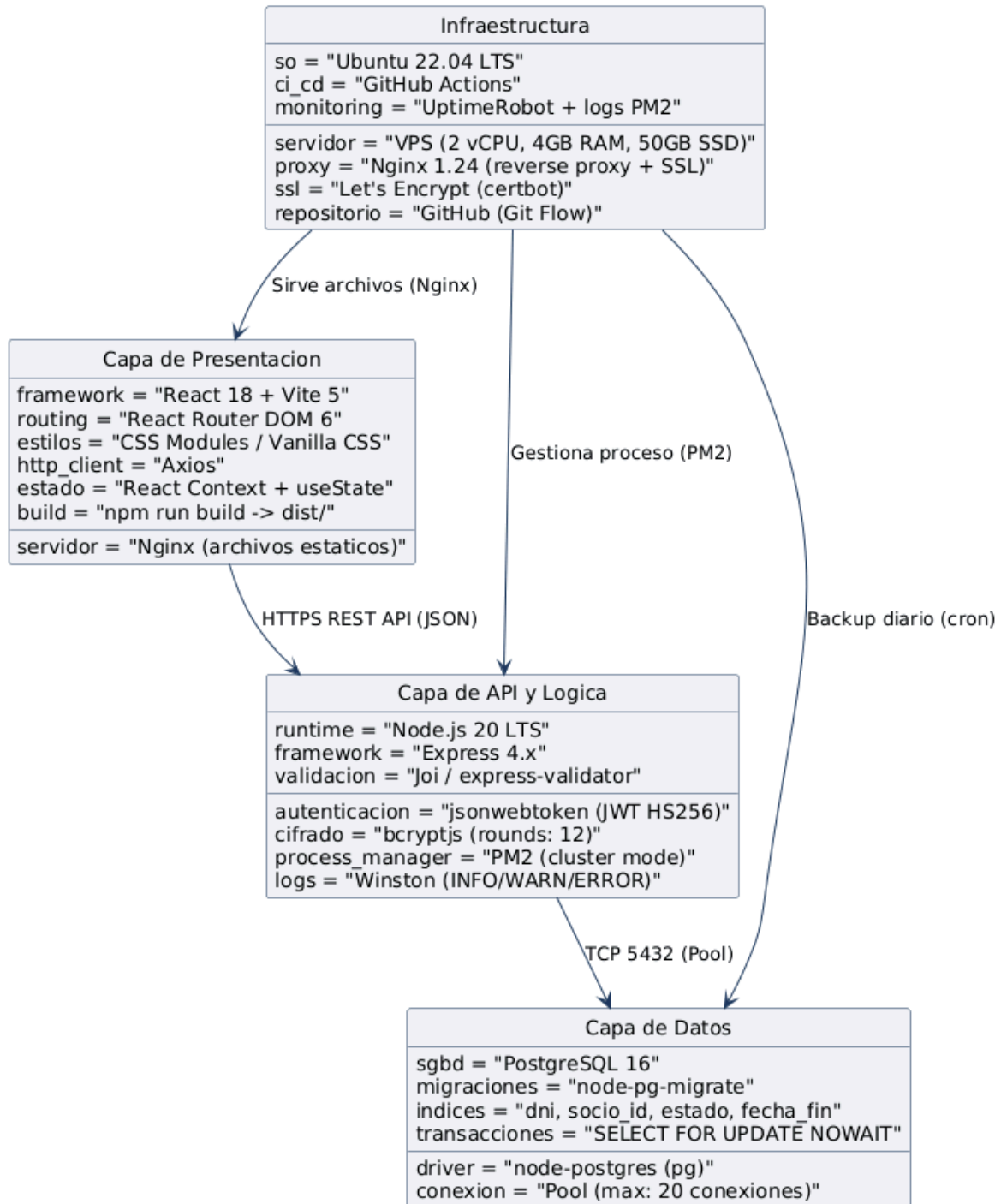


Figura 2.1: Diagrama de Clases — Stack Tecnológico de Producción OLYMPUS CORE

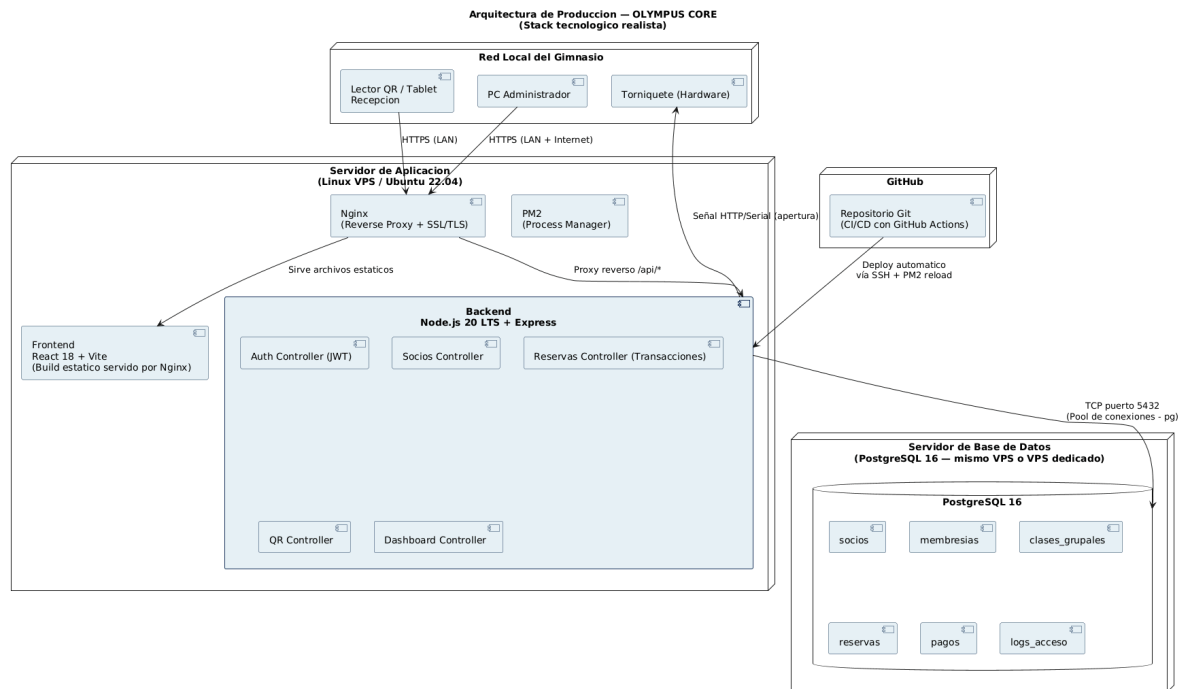


Figura 2.1b: Diagrama de Componentes — Arquitectura de Producción (VPS + Nginx + PostgreSQL)

2.2. Análisis de Riesgos

Restricción	Riesgo Identificado	Probabilidad	Impacto	Mitigación
Límite de tiempo (5 meses)	No completar el alcance total de los módulos operativos del MVP1 en el tiempo establecido.	Alta	Alto	Usar Planning Game de XP para priorizar el MVP1. Ciclos semanales para asegurar valor temprano. Matriz MoSCoW congelada.
Metodología XP (TDD y Pair Programming)	Resistencia del equipo a programar en pares o curva de aprendizaje lenta al escribir pruebas antes que el código.	Media	Medio	Rotación constante de parejas para compartir conocimiento. Mentoría técnica y adopción gradual del TDD en los componentes

				más críticos primero.
Base de datos PostgreSQL	Inconsistencias en el esquema de BD debido a cambios rápidos y refactorización propia del ciclo ágil.	Baja	Alto	Control de versiones de la BD con node-pg-migrate integrado al pipeline de CI/CD. Cada cambio al esquema requiere un archivo de migración aprobado.
Integración QR con hardware del gimnasio	Incompatibilidad del protocolo del torniquete físico actual con el sistema web (HTTP vs. serial RS-232).	Media	Alto	Desarrollar un spike (prototipo arquitectónico) en la semana 1 para validar la comunicación con el torniquete. Definir protocolo (HTTP o adaptador serial) antes de la implementación.
Seguridad (JWT + bcrypt)	Robo de credenciales, tokens JWT comprometidos o accesos indebidos por mal manejo de secretos.	Media	Crítico	Variables de entorno gestionadas con dotenv + archivos .env.production en el servidor (nunca en el repositorio). JWT con expiración estricta. Rotación de secretos en caso de compromiso.
Escalabilidad bajo carga	Lentitud o caída del sistema ante pico de usuarios (hora pico 7AM y 6PM con 100+)	Media	Alto	Arquitectura modular con PM2 en modo cluster (usa todos los núcleos del VPS).

	socios concurrentes).			Índices en PostgreSQL en columnas críticas. Pruebas de carga con k6 antes del go-live.
APIs externas (SC-001, SC-002)	Caída del servicio de WhatsApp Business API o pasarela de pagos durante horas de operación.	Baja	Medio	Patrón retry con exponential backoff (máximo 3 intentos). Canal de respaldo (correo electrónico). Las fallas de APIs externas no deben bloquear el flujo principal del MVP1.
Infraestructura VPS	Caída del servidor físico o agotamiento de recursos (CPU/RAM/disco) en momentos críticos.	Baja	Alto	Monitoreo con UptimeRobot (ping cada 5 min). Alertas por email al equipo. Backups diarios automáticos de la BD con pg_dump + cron. Plan de recuperación documentado.

3. Especificación de Requisitos de Desarrollo

Después de la identificación y análisis de restricciones, el equipo documentó detalladamente los requisitos técnicos necesarios para construir una plataforma robusta, escalable y alineada a los objetivos del gimnasio.

3.1. Requisitos de Desarrollo

ID	Categoría	Descripción
RD-DEV-01	Frontend	El frontend será desarrollado con React 18 y Vite 5. La UI será construida con CSS Modules/Vanilla CSS. La comunicación con el backend se realizará con Axios sobre HTTPS. El build de producción genera archivos estáticos servidos por Nginx.
RD-DEV-02	Backend	El backend será implementado con Node.js 20 LTS y Express 4.x utilizando arquitectura basada en APIs REST versionadas (/api/v1/). Se gestionará con PM2 en modo cluster para aprovechar todos los núcleos del VPS.
RD-DEV-03	Base de Datos	PostgreSQL 16 será el SGBD. Se usará el driver node-postgres (pg) con pool de conexiones (máximo 20). Las migraciones se gestionarán con node-pg-migrate. Se crearán índices en columnas de búsqueda frecuente (dni, socio_id, estado, fecha_fin).
RD-DEV-04	Seguridad	El sistema implementará autenticación con JWT (HS256, exp: 24h para sesiones, 12h para QR) y cifrado bcrypt (factor: 12 rondas). Todas las rutas protegidas requerirán el header Authorization: Bearer <token>. Control de acceso RBAC por middleware.
RD-DEV-05	Concurrencia	El módulo de reservas implementará transacciones PostgreSQL con SELECT FOR UPDATE NOWAIT para prevenir condiciones de

		carrera (Race Conditions) en la actualización del aforo. El sistema debe soportar 100 usuarios concurrentes sin sobreventa de cupos.
RD-DEV-06	QR y Acceso	El módulo de validación de acceso generará tokens QR con JWT firmado (exp: 12h). El endpoint POST /api/acceso/validar validará la firma, membresía y penalizaciones en < 300 ms (P95) y registrará cada intento en la tabla logs acceso.
RD-DEV-07	Infraestructura	El servidor de producción será un VPS Linux (Ubuntu 22.04). Nginx actuará como reverse proxy con SSL/TLS (Let's Encrypt). El despliegue se automatizará con GitHub Actions vía SSH. No se utilizarán plataformas PaaS gratuitas con limitaciones de inactividad.
RD-DEV-08	Control de versiones	El equipo utilizará GitHub con Git Flow: ramas main (producción), develop (integración) y feature/* (funcionalidades). Cada PR requiere revisión de al menos 1 compañero antes del merge.
RD-DEV-09	Metodología	El desarrollo seguirá XP: Pair Programming rotativo, TDD para módulos críticos (reservas, QR, autenticación), sprints de 1 semana con Planning Game, cliente in-situ (o representante) para feedback continuo.
RD-DEV-10	CI/CD	El pipeline de GitHub Actions ejecutará: lint (ESLint), tests unitarios (Jest + Supertest), tests de integración, build de producción y deploy automático al VPS vía SSH con PM2 reload (zero-downtime). El pipeline incluye health check

		post-deploy con rollback automático.
RD-DEV-11	Pruebas	Se implementarán pruebas unitarias (Jest) para controladores y servicios, pruebas de integración (Supertest) para endpoints REST, y pruebas de carga con k6 para validar RNF-01, RNF-02 y RNF-03 antes del go-live.
RD-DEV-12	Rendimiento	El sistema debe mantener tiempos de respuesta: POST /acceso/validar < 300 ms (P95), endpoints generales < 500 ms (P95), bajo carga de 100 VUs concurrentes. Se implementarán índices en PostgreSQL y caché a nivel de aplicación donde aplique.
RD-DEV-13	Logs y Monitoreo	Se implementará logging estructurado con Winston (niveles INFO/WARN/ERROR) con rotación diaria de archivos. El monitoreo de disponibilidad (uptime) se realizará con UptimeRobot. Los logs de acceso QR se almacenarán en la tabla logs_acceso de PostgreSQL.
RD-DEV-14	Backups	Se configurará un job cron diario (3:00 AM) para ejecutar pg_dump de la BD de producción. Los backups se almacenarán en una ruta segura del VPS con retención de 30 días. En caso de falla crítica, se podrá restaurar en < 4 horas.

3.2. Requisito Técnico TEC-001

ID: TEC-001

ID	TEC-001
Título	Implementar módulo de reservas de clase grupal con control de concurrencia
Tipo	Tarea técnica (backend + concurrencia + BD)

RF asociados	RF-09 (Reservar clase grupal), RF-14 (Penalización), RF-16 (Gestión de clases), RF-18 (Control de aforo)
RNF asociados	RNF-01 (Latencia < 500ms), RNF-03 (100 VUs concurrentes)
RQ-CAL asociados	RQ-CAL-OLY-02 (Latencia API), RQ-CAL-OLY-03 (Capacidad)

Se necesita la implementación de un módulo backend que permita a los socios reservar clases grupales en tiempo real, validando automáticamente la disponibilidad de cupos, membresía activa y restricciones por penalización. El módulo debe soportar múltiples reservas simultáneas sin pérdida ni sobrescritura de información, implementando transacciones PostgreSQL con bloqueos a nivel de fila (SELECT FOR UPDATE NOWAIT) para prevenir condiciones de carrera (Race Conditions).

Criterios de aceptación (funcionales):

- Dado un usuario con membresía activa, sin penalización activa y cupos disponibles, el sistema registra la reserva correctamente (HTTP 201).
- Después de cada reserva exitosa, el aforo_actual de la clase se actualiza automáticamente en la BD.
- Si la clase no tiene cupos disponibles, el sistema devuelve HTTP 409 con mensaje CLASE_SIN_CUPOS.
- El sistema bloquea la reserva si el socio tiene membresía vencida (HTTP 400 MEMBRESIA_VENCIDA).
- El sistema bloquea la reserva si el socio tiene penalización activa (HTTP 400 PENALIZACION_ACTIVADA).
- El sistema rechaza reservas duplicadas en el mismo horario (HTTP 409 RESERVA_DUPLICADA).
- Se registran todas las reservas en la BD con fecha, hora y usuario asociado.
- El sistema soporta reservas simultáneas sin duplicidad ni inconsistencia (verificado con prueba de carga).

Requisitos no funcionales:

- Rendimiento: El proceso de reserva debe completarse en < 500 ms (P95) bajo carga de 100 VUs.
- Concurrencia: El sistema debe soportar al menos 100 usuarios simultáneos sin sobreventa de cupos.
- Disponibilidad: El módulo debe mantenerse operativo el 99% del tiempo (RQ-CAL-OLY-11).
- Integridad: No debe existir sobrescritura ni pérdida de reservas. Transacciones ACID garantizadas.
- Compatibilidad: Los endpoints REST deben ser consumibles desde el cliente web y (en v3.0) desde la app móvil.

Dependencias técnicas:

- PostgreSQL 16 configurado con soporte transaccional y row-level locking.
- Módulo de autenticación JWT operativo (middleware auth).
- Módulo de penalizaciones operativo (RF-14 implementado).
- Horarios y aforos previamente registrados en la BD.
- Índice en clases_grupales(id) para garantizar que el SELECT FOR UPDATE use Index Scan.

El siguiente diagrama ilustra el control de concurrencia en el módulo de reservas:

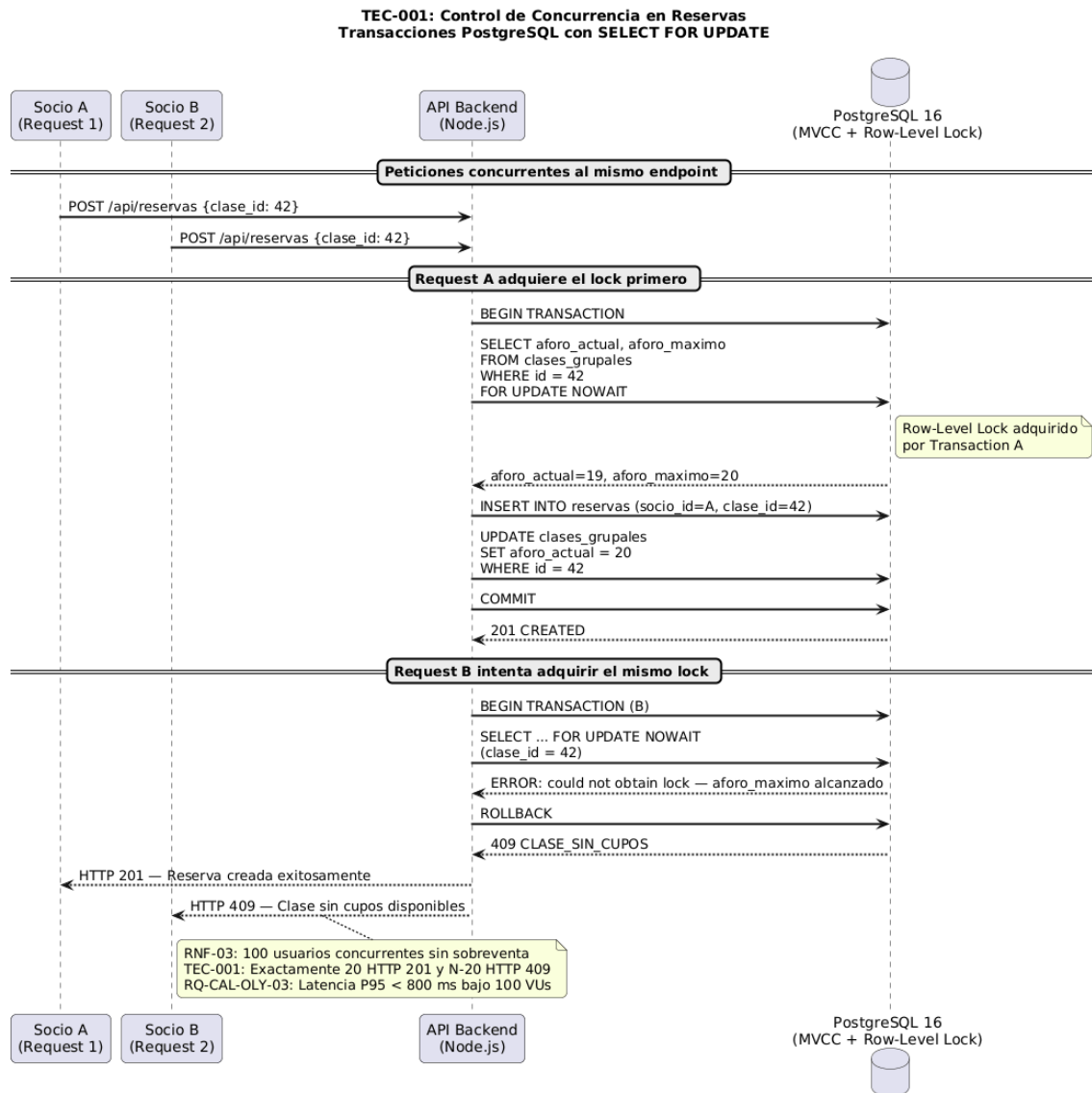


Figura 3.2: Diagrama de Secuencia — Control de Concurrencia en Reservas con SELECT FOR UPDATE (TEC-001)

Definition of Ready (DoR):

Criterio	Estado requerido	Responsable
----------	------------------	-------------

Modelo de datos validado	Tablas clases grupales y reservas implementadas con índices	Backend Team
APIs REST definidas	Endpoints documentados (POST /api/reservas, etc.)	Backend Team
Reglas de negocio definidas	Penalizaciones, aforos y cancelaciones documentadas	QA Team
Diagramas UML disponibles	Casos de uso y secuencias completados	Equipo de Análisis
Casos de prueba definidos	Escenarios concurrentes y de error documentados	QA Team
Entorno backend operativo	Servidor VPS, PostgreSQL y API base funcionando	DevOps / Backend

Definition of Done (DoD):

Criterio	Verificación
Reservas operativas	Registro correcto de reservas con todos los campos requeridos
Validación de aforo	aforo_actual se actualiza automáticamente tras cada reserva exitosa
Manejo concurrente	Prueba de carga k6: 0 reservas duplicadas con 100 VUs simultáneos
Pruebas Jest/Supertest OK	Todos los casos de prueba (éxito, cupo lleno, membresía vencida) pasan
Logs implementados	Tabla reservas con trazabilidad completa (socio_id, clase_id, timestamp)
Seguridad validada	Endpoint protegido por middleware JWT, solo socios autenticados pueden reservar
Documentación actualizada	Endpoints documentados con ejemplos de request/response
Revisión técnica	Code review aprobado por al menos 1 integrante del equipo

3.3. Requisito Técnico TEC-002

ID	TEC-002
Título	Implementar módulo de validación de acceso mediante código QR dinámico
Tipo	Tarea técnica (backend + integración QR + hardware)
RF asociados	RF-06 (Validar acceso QR), RF-07 (Dashboard con logs), RF-10 (Ver perfil y QR), RD-01, RD-12
RNF asociados	RNF-02 (QR < 300ms), RNF-05 (HTTPS), RNF-06 (JWT exp.)
RQ-CAL asociados	RQ-CAL-OLY-01 (Latencia QR), RQ-CAL-OLY-06 (Expiración token)

Implementación de un servicio backend que reciba tokens QR generados desde la aplicación del socio (JWT firmado con exp de 12 horas), consulte el estado de la membresía en tiempo real en PostgreSQL y permita o bloquee automáticamente el acceso al gimnasio mediante integración con el sistema de torniquete físico. El módulo debe registrar cada intento de acceso en logs estructurados para fines de auditoría y trazabilidad, y responder en < 300 ms (P95) para no generar colas en recepción.

Criterios de aceptación (funcionales):

- Dado un token QR válido (firma JWT correcta, no expirado) y membresía activa sin penalización: el sistema devuelve HTTP 200 {resultado: "PERMITIDO"} y señala apertura del torniquete.
- Dado un token QR expirado o con firma inválida: el sistema devuelve HTTP 401 {motivo: "TOKEN_INVALIDO"} sin consultar la BD.
- Dado un QR válido pero membresía vencida: el sistema devuelve HTTP 403 {motivo: "MEMBRESIA_VENCIDA"}.
- Dado un QR válido pero penalización de bloqueo activa: el sistema devuelve HTTP 403 {motivo: "PENALIZACION_ACTIVADA", desbloqueado_en: "<fecha>"}
- El sistema registra socio_id, fecha/hora, resultado y motivo en la tabla logs_acceso para cada intento.
- Dado un error de conexión a BD o timeout (> 5s): el sistema reintenta hasta 2 veces antes de devolver HTTP 503 {error: "SERVICE_UNAVAILABLE"}.
- El sistema registra logs estructurados (INFO, WARN, ERROR) para cada intento de acceso usando Winston.

Requisitos no funcionales:

- Seguridad: tokens JWT firmados con HS256. Secreto almacenado en variable de entorno (nunca en código fuente). bcrypt para contraseñas (RNF-04).
- Rendimiento: el tiempo promedio de validación completa (JWT → BD → respuesta) debe ser < 300 ms (P95) — RQ-CAL-OLY-01.
- Disponibilidad: el módulo debe mantenerse operativo el 99% del tiempo en horario operativo del gimnasio — RQ-CAL-OLY-11.
- Trazabilidad: todos los accesos (permitidos y denegados) deben almacenarse en logs_acceso con campos: socio_id, timestamp, resultado, motivo — RD-09.
- Escalabilidad: el sistema debe soportar más de 100 validaciones concurrentes sin degradación — RNF-03.

Dependencias técnicas:

- PostgreSQL 16 operativo con índices en socios(id), membresias(socio_id, estado, fecha_fin).
- Integración validada con lector QR y torniquete físico del gimnasio (protocolo HTTP o adaptador serial).

- Variable de entorno JWT_SECRET definida en el servidor de producción (no en el repositorio).
- Conexión estable del servidor VPS a la red local del gimnasio (LAN o VLAN dedicada).
- Certificado SSL/TLS válido instalado en Nginx.

El siguiente diagrama muestra los estados posibles del token QR durante la validación:

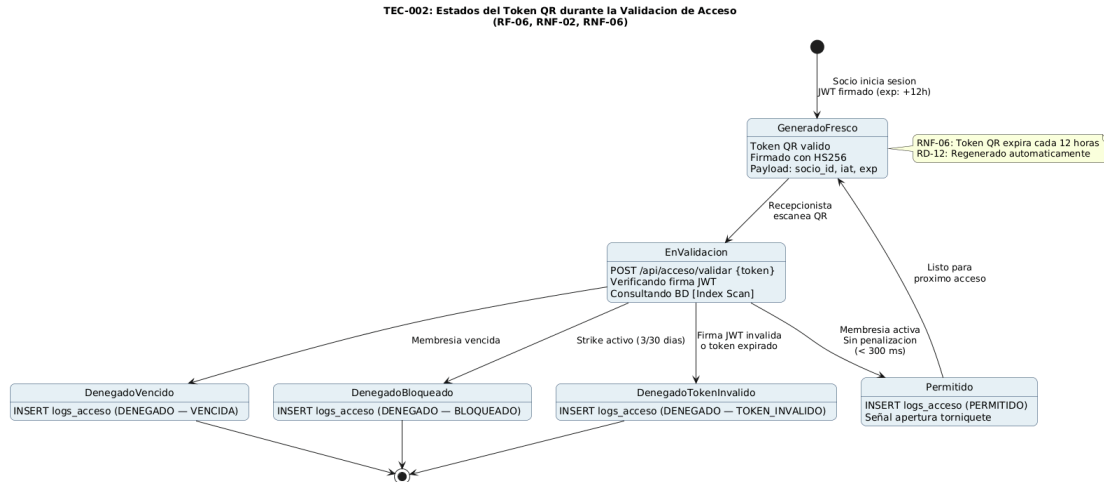


Figura 3.3: Diagrama de Estado — Ciclo de Vida del Token QR (TEC-002, RF-06, RNF-02, RNF-06)

Definition of Ready (DoR):

Criterio	Estado requerido
Diagramas UML disponibles	Diagramas de secuencia, estado y actividades completados
Base de datos modelada	Tablas socios, membresias y logs_acceso con índices validados
Endpoints definidos	POST /api/acceso/validar documentado con esquema de request/response
Validaciones definidas	Reglas de membresía, expiración QR y penalizaciones documentadas
Entorno de desarrollo	Backend y PostgreSQL configurados en VPS o entorno local equivalente
Casos de prueba definidos	Escenarios de éxito, QR expirado, membresía vencida y penalización documentados
Seguridad documentada	JWT_SECRET y bcrypt rounds especificados. Variables de entorno definidas.
Integración hardware validada	Spike de conexión con torniquete completado (protocolo confirmado)

Definition of Done (DoD):

Criterio	Verificación
Código implementado	Funcionalidad integrada y desplegada en el VPS de producción
Validación QR operativa	Acceso PERMITIDO y DENEGADO funcionando con logs correctos
Pruebas Jest/Supertest OK	Todos los escenarios definidos en el DoR pasan exitosamente
Logs implementados	tabla logs_acceso poblada correctamente en pruebas de integración
Manejo de errores	Timeout y BD no disponible manejados con retry + HTTP 503
Seguridad validada	JWT verificado, bcrypt funcional, rutas sin JWT bloqueadas con 401
Documentación técnica	Endpoint documentado con curl/Postman. Variables de entorno documentadas.
Revisión de código	Code review aprobado por al menos 1 integrante del equipo

3.4. Requisito Técnico TEC-003

ID	TEC-003
Título	Configuración del pipeline CI/CD y entorno de producción (VPS + Nginx + PM2)
Tipo	Tarea técnica (DevOps + infraestructura)
RF asociados	Todos los RF — es la infraestructura que los soporta
RNF asociados	RNF-11 (Disponibilidad 99%), RNF-12 (Control de versiones), RNF-13 (Índices BD)
RQ-CAL asociados	RQ-CAL-OLY-11 (Uptime), RQ-CAL-OLY-12 (Git), RD-DEV-07, RD-DEV-10

Configuración completa del entorno de producción en un VPS Linux y del pipeline de integración y despliegue continuo (CI/CD) con GitHub Actions. El objetivo es garantizar que cada push a la rama main dispare automáticamente lint, tests, build y deploy con zero-downtime, incluyendo health check y rollback automático en caso de fallo.

Criterios de aceptación:

- El pipeline se ejecuta automáticamente en cada push a la rama main.
- Un fallo en lint, tests o integración detiene el pipeline antes del deploy.
- El deploy utiliza SSH + PM2 reload para garantizar zero-downtime (el sistema no deja de responder durante el despliegue).
- Tras el deploy, el pipeline verifica GET /api/health → HTTP 200 antes de marcar el deploy como exitoso.
- Si el health check falla, el pipeline ejecuta rollback automático al commit anterior.

- Los certificados SSL/TLS se gestionan con Let's Encrypt (certbot) y se renuevan automáticamente.
- Los backups de PostgreSQL se ejecutan diariamente a las 3:00 AM con pg_dump.

El siguiente diagrama muestra el pipeline CI/CD completo:

**Pipeline CI/CD — GitHub Actions
OLYMPUS CORE**

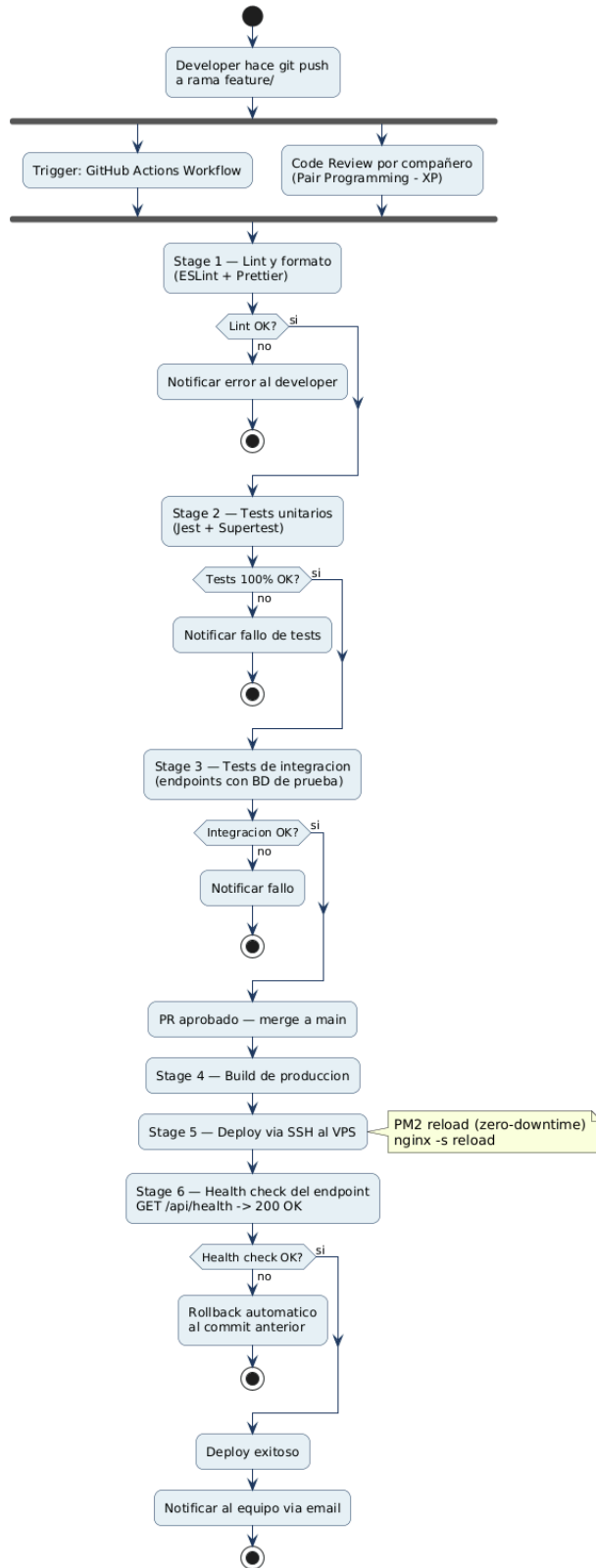


Figura 3.4: Diagrama de Actividad — Pipeline CI/CD con GitHub Actions (TEC-003)

4. Verificación de Requisitos de Desarrollo

4.1. Estrategias Generales de Validación

Revisiones de Pares (Peer Reviews): Se realizarán inspecciones técnicas estáticas mediante grupos de desarrolladores para validar que los requisitos documentados (DoR y DoD) sean comprendidos unánimemente. Cada PR en GitHub requiere aprobación de al menos 1 integrante antes del merge.

Prototipos Arquitectónicos (Spikes): Antes de programar el sistema completo, se desarrollará un prototipo rápido del escaneo del código QR y su comunicación con el torniquete físico para validar la compatibilidad del protocolo (HTTP o serial RS-232). El spike se completará en la Semana 1 del proyecto.

Pruebas de Estrés y Rendimiento: Se ejecutarán simulaciones de carga concurrente con k6 en el entorno de desarrollo para asegurar que el módulo de reservas soporte 100 VUs sin sobreventa (TEC-001) y que la validación QR responda en < 300 ms (TEC-002). Las pruebas se corren antes del go-live.

Auditorías de Seguridad (OWASP): Se ejecutarán pruebas de vulnerabilidades basadas en las guías OWASP Top 10 para validar que los datos personales (ficha médica, DNI), contraseñas y tokens estén correctamente protegidos. Se probará específicamente SQL Injection y XSS (RQ-CAL-OLY-07).

Verificación Automatizada (CI/CD): Se auditará el cumplimiento de los estándares de codificación insertando ESLint y Jest directamente en el pipeline de GitHub Actions. Ningún código puede llegar a producción sin pasar lint y pruebas automatizadas.

Pruebas de Disponibilidad: Se configurará UptimeRobot para hacer ping al endpoint /api/health cada 5 minutos durante 30 días. La disponibilidad calculada debe ser >= 99% en horario operativo (L-S 6AM-10PM) para cumplir RQ-CAL-OLY-11.

4.2. Inspección Técnica

Requisito Original (ambiguo — extraído del documento original):

"El sistema backend debe registrar las reservas de las clases de spinning instantáneamente y bloquear la base de datos de forma segura para que no haya sobrecupos, soportando a todos los usuarios conectados a las 6:00 p.m."

Evaluación del requisito original:

Atributo	Evaluación	Observaciones	Recomendación
Atomicidad	No atómico	Mezcla dos preocupaciones distintas: (a) el tiempo de respuesta	Separar ambos conceptos en TEC-001 (conurrencia) y

		del sistema ("instantáneamente") y (b) el control de concurrencia y sobrecupos en la base de datos. Deben especificarse en requisitos separados.	TEC-002 (latencia QR).
Testabilidad	No testable	Palabras como "instantáneamente" y "todos los usuarios" no son cuantificables. El equipo de pruebas no puede medir "todos"; necesita un número máximo de peticiones simultáneas (ej. 100 req/seg) para configurar la prueba de estrés con k6.	Definir métricas cuantificables: 100 VUs concurrentes, P95 < 500 ms.
Factibilidad	Parcialmente factible	Node.js y PostgreSQL pueden manejar concurrencia, pero "bloquear la base de datos entera" no es factible ni eficiente. Bloquearía todos los módulos del sistema durante la reserva.	Especificar bloqueo a nivel de fila con SELECT FOR UPDATE NOWAIT en la tabla clases_grupoales.
Ambigüedad	Ambiguo	Términos como "de forma segura" son vagos. ¿Se refiere a seguridad contra ataques (OWASP — RQ-CAL-OLY-07) o a seguridad transaccional (ACID)?	Especificar explícitamente: transacciones ACID de PostgreSQL para integridad y JWT/bcrypt para seguridad de acceso.

Requisito Técnico Reescrito y Validado (TEC-003 reescrito):

Al detectar las deficiencias y la falta de parámetros matemáticos para las pruebas automatizadas, el requisito original se reescribe y divide en dos requisitos técnicos medibles:

ID reescrito	TEC-001 (Control de concurrencia en reservas)
---------------------	---

Descripción	El endpoint POST /api/reservas debe implementar transacciones PostgreSQL con SELECT FOR UPDATE NOWAIT para prevenir condiciones de carrera (Race Conditions) al verificar y actualizar el aforo máximo de una clase grupal.
Criterio matemático de aceptación	Una prueba de carga con k6 simulará 150 peticiones concurrentes en el mismo segundo apuntando a una clase con aforo máximo de 20 socios. Condición de éxito: exactamente 20 respuestas HTTP 201 y 130 respuestas HTTP 409, con P95 < 500 ms para todo el lote.

Validaciones técnicas esperadas para TEC-001:

Validación	Resultado esperado
Control concurrente	No existe sobreventa de cupos (exactamente 20 reservas creadas)
Integridad de datos	No existen reservas duplicadas para el mismo socio+clase
Transacciones PostgreSQL ACID	SELECT FOR UPDATE NOWAIT operativo — verificado con EXPLAIN ANALYZE
Respuestas HTTP diferenciadas	HTTP 201 (éxito) y HTTP 409 (sin cupos) correctamente devueltos
Rendimiento bajo carga	P95 <= 500 ms para el lote completo de 150 peticiones concurrentes
Rollback en caso de error	Ninguna reserva queda en estado inconsistente si la transacción falla

Riesgos mitigados por TEC-001:

- Pérdida de integridad del aforo (sobreventa de cupos de clase).
- Inconsistencias concurrentes entre múltiples socios que reservan simultáneamente.
- Corrupción de datos en la tabla clases_grupales.aforo_actual.
- Errores de sincronización entre usuarios simultáneos en hora pico (6:00 PM).

5. Conclusiones

- Los requisitos de desarrollo permitieron identificar restricciones técnicas, metodológicas, operativas y legales críticas para el proyecto OLYMPUS CORE, documentándolas de forma sistemática y alineándolas con los 34 RF, 13 RNF y 13 RQ-CAL del sistema.
- La selección de un stack tecnológico realista (Node.js 20 + PostgreSQL 16 + React 18 + Nginx en VPS Linux) garantiza que el sistema pueda operar en producción con alta disponibilidad, seguridad y rendimiento, sin depender de plataformas con limitaciones de recursos o tiempo de inactividad.
- La aplicación de metodologías ágiles (XP) combinadas con CI/CD automatizado (GitHub Actions) mejora la calidad, trazabilidad y velocidad del desarrollo, permitiendo entregables funcionales al final de cada sprint semanal.
- La verificación técnica aplicada al requisito ambiguo del documento original permitió transformar un requisito no testable en dos requisitos técnicos medibles (TEC-001 y TEC-002) con métricas matemáticas precisas verificables con herramientas de prueba de carga (k6).
- La implementación de control concurrente con SELECT FOR UPDATE NOWAIT en PostgreSQL, integración de código QR con JWT de expiración dinámica y seguridad basada en bcrypt + HTTPS fortalece la confiabilidad, seguridad y escalabilidad del sistema de cara a los 500 socios activos del gimnasio.
- El enfoque incremental basado en MVP1 facilita la construcción progresiva del sistema y reduce riesgos durante el desarrollo, sentando las bases técnicas necesarias para las versiones 2.0 (pagos online, WhatsApp) y 3.0 (app móvil, módulo nutricional) identificadas en el proceso de gestión de cambios.